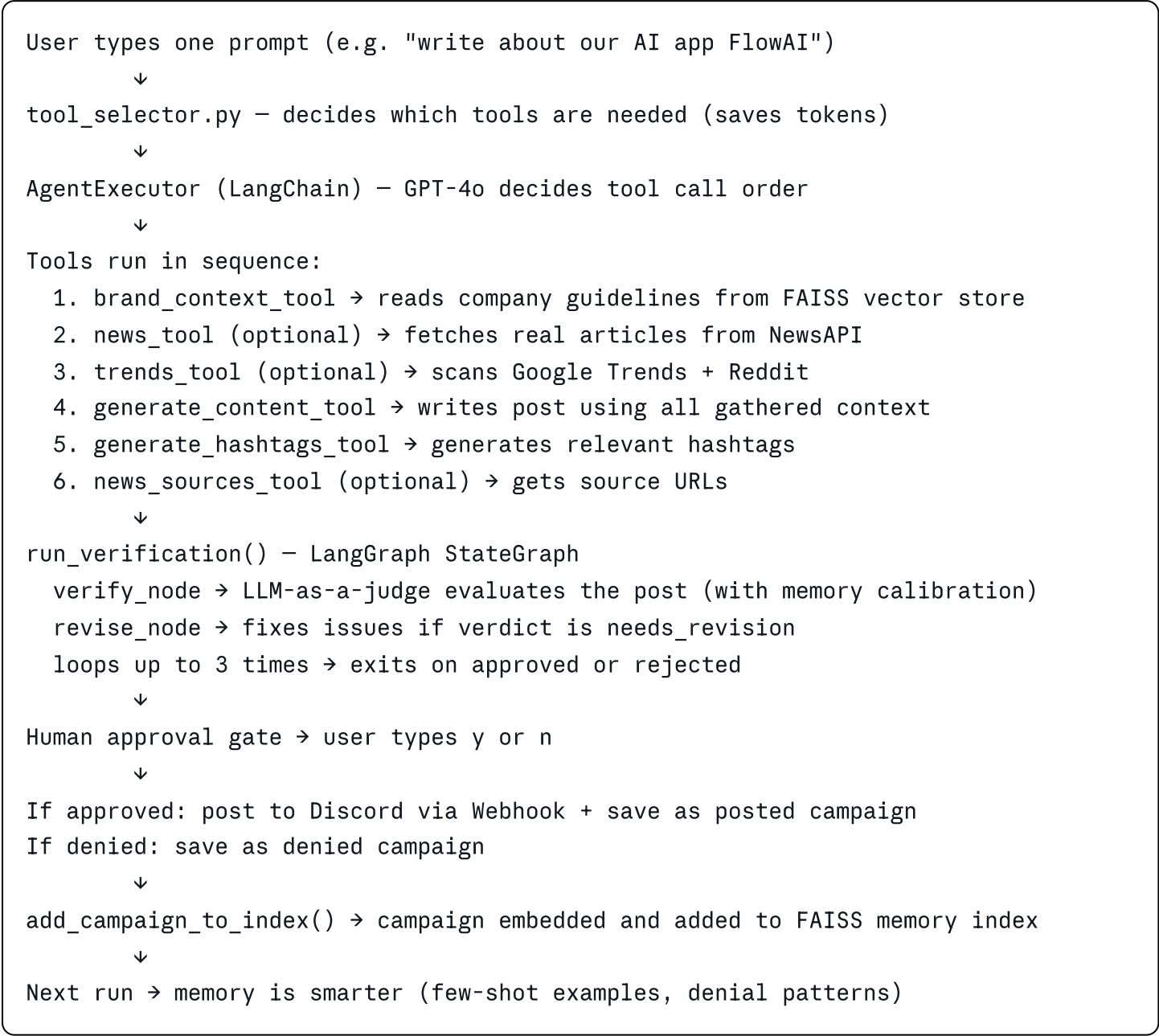


# Presentation Preparation — Marketing Agent (Phase 1-3)

---

## 1. General Flow of the Project



## 2. Technologies Used

| Technology | What it is                       | Where it is used                                                      |
|------------|----------------------------------|-----------------------------------------------------------------------|
| OpenAI API | GPT-4o and GPT-4o-mini LLM calls | Agent reasoning, content generation, hashtags, verification, revision |
| LangChain  | AI agent orchestration framework | AgentExecutor, tool definitions, prompt templates                     |

| Technology                  | What it is                                | Where it is used                                            |
|-----------------------------|-------------------------------------------|-------------------------------------------------------------|
| <b>LangGraph</b>            | State machine for cyclic LLM workflows    | verify → revise loop in verify.py                           |
| <b>LangSmith</b>            | Observability and tracing for LLM apps    | Automatic tracing of every LLM call                         |
| <b>FAISS</b>                | Vector similarity search library          | Brand RAG (rag.py) and campaign memory (campaign_memory.py) |
| <b>OpenAI Embeddings</b>    | text-embedding-3-small model              | Converts text into semantic vectors for search              |
| <b>NewsAPI</b>              | REST API for recent news articles         | news.py — fetch real articles for grounded content          |
| <b>SerpAPI</b>              | Google Search API                         | trends.py — fetch Google Trends signals                     |
| <b>Reddit (public JSON)</b> | Reddit's public search endpoint           | trends.py — fetch hot Reddit discussions                    |
| <b>Discord Webhook</b>      | HTTP POST endpoint for posting to Discord | discord.py — deliver final approved content                 |
| <b>python-dotenv</b>        | Load environment variables from .env file | All files that need API keys                                |
| <b>tiktoken</b>             | OpenAI tokenizer                          | tool_selector.py — count and report token savings           |
| <b>requests</b>             | Python HTTP library                       | Discord, NewsAPI, SerpAPI calls                             |

### 3. What I Learnt

**LangChain and agent architecture** How to structure an AI agent that doesn't just call one LLM — it orchestrates multiple tool calls, routes decisions, and builds a conversation history. The difference between a simple LLM call and a proper agent with tool use.

**LangGraph** How to model a looping workflow (verify → revise → verify) as a directed graph with nodes, edges, and conditional routing. Why a state machine is better than a while loop for this — state is explicit, transitions are traceable, and LangSmith can inspect every step.

**RAG (Retrieval-Augmented Generation)** How to give an LLM access to private knowledge (brand guidelines, company data) without fine-tuning it. The pipeline: chunk text → embed → store in vector store → retrieve top-k at query time → inject into prompt.

**Embeddings and vector search** What embeddings actually are (dense vectors representing semantic meaning), how FAISS stores and searches them using approximate nearest-neighbour algorithms, and why semantic search finds relevant results that keyword search would miss.

**LLM-as-a-judge** A technique where one LLM call evaluates the output of another LLM call. The judge uses temperature=0 for consistency, structured JSON output for parsability, and reference examples for calibration. This is how automated content quality gates work in production.

**Few-shot prompting** Showing the model concrete examples of good output before asking it to generate — significantly more effective than detailed instructions alone. The model implicitly learns style, tone, and structure from examples rather than being told about them.

**Token economics** Every character in a prompt costs money and adds latency. Tool descriptions, system prompts, conversation history, and retrieved documents all consume tokens. Designing the system to pass only what's necessary is a real engineering skill.

**Memory in AI systems** The difference between short-term memory (conversation context, disappears after session) and long-term memory (persisted to disk, searchable across sessions). How vector search turns a folder of JSON files into a queryable knowledge base.

**Feedback loops** How to capture approval/denial signals from the user and feed them back into the system as training data — not model fine-tuning, but structured examples that calibrate future decisions. This is a lightweight form of preference learning.

---

## 4. Essential Functions and Tools

### Core agent functions

| Function                          | File             | What it does                                                                                                        |
|-----------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>run_agent()</code>          | agent.py         | Orchestrates the full generation: selects tools, retrieves few-shot examples, runs AgentExecutor, extracts articles |
| <code>select_tools()</code>       | tool_selector.py | Fast GPT-4o-mini call to decide which of the 6 tools to use. Saves tokens                                           |
| <code>parse_agent_output()</code> | agent.py         | Splits raw agent output into content / hashtags / sources                                                           |
| <code>run_verification()</code>   | verify.py        | Invokes the LangGraph verify↔revise StateGraph with memory calibration                                              |
| <code>run_campaign()</code>       | main.py          | Top-level function: agent → verify → human approval → save → index                                                  |

| Function                                              | File                            | What it does                                                          |
|-------------------------------------------------------|---------------------------------|-----------------------------------------------------------------------|
| <code>retrieve_brand_context()</code>                 | <code>rag.py</code>             | Semantic search over <code>company_data.json</code> sections          |
| <code>get_few_shot_examples()</code>                  | <code>campaign_memory.py</code> | Find similar approved campaigns by prompt<br>→ inject into copywriter |
| <code>get_denied_examples()</code>                    | <code>campaign_memory.py</code> | Find similar denied campaigns by content<br>→ warn verifier           |
| <code>get_approved_examples_for_verification()</code> | <code>campaign_memory.py</code> | Find similar approved campaigns by content<br>→ calibrate verifier    |
| <code>add_campaign_to_index()</code>                  | <code>campaign_memory.py</code> | Embed new campaign and add to FAISS index                             |

## LangGraph nodes

| Node                            | What it does                                                                                      |
|---------------------------------|---------------------------------------------------------------------------------------------------|
| <code>verify_node</code>        | LLM-as-a-judge: evaluates content against rules + past examples. Returns verdict JSON             |
| <code>revise_node</code>        | Copy editor: rewrites content to fix the specific issues <code>verify_node</code> found           |
| <code>route_after_verify</code> | Conditional edge function: approved → END, rejected → END, needs_revision → revise (if under cap) |

## 5. What Each Code File Does

|                       |                                                                                 |
|-----------------------|---------------------------------------------------------------------------------|
| <code>main.py</code>  | – Entry point. CLI loop. Orchestrates the 3-step flow.                          |
| <code>agent.py</code> | – Builds <code>AgentExecutor</code> . Retrieves few-shot memory. Parses output. |

```

services/
  ai.py          - generate_content() and generate_hashtags(). Reads few-shot
cache.
  discord.py     - post_to_discord(). Sends HTTP POST to Discord Webhook.
  news.py        - fetch_news(). Calls NewsAPI. Caches last fetched articles.
  news_integration.py - fetch_articles_for_marketing(). Deduplication + formatting
adapter.
  trends.py      - fetch_google_trends() + fetch_reddit_trends(). SerpAPI +
Reddit.
  storage.py     - save_campaign(). Writes JSON to campaigns/ with status +
verdict info.
  tools.py       - All 6 LangChain @tool decorated functions. Docstrings =
tool descriptions.
  tool_selector.py - select_tools(). GPT-4o-mini pre-call to pick relevant
tools.
  rag.py         - Loads company_data.json, chunks it, embeds into FAISS,
retrieves context.
  verify.py      - LangGraph StateGraph. VerificationState. verify_node +
revise_node.
  llm.py         - Single shared ChatOpenAI instance (GPT-4o-mini, temp=0.7).
  campaign_memory.py - FAISS index for campaigns. Build, load, search, add
functions.

company_data.json - Mock brand dataset. Brand, products, tone, approved
claims, forbidden phrases.
campaigns/        - Every run saved as posted_TIMESTAMP.json or
denied_TIMESTAMP.json.
memory/           - FAISS index files (index.faiss + index.pkl) for campaign
memory.
.env              - API keys. Never committed to git.

```

## 6. What Is Happening When the Terminal Prints Output

```

=== Marketing Agent - AI Powered ===

Initializing campaign memory...
[Memory] Campaign index loaded - 12 campaigns

```

→ At startup, FAISS index is loaded from memory/ folder. 12 past campaigns are ready for search.

```

What do you want to post about?
→ Launch our AI app FlowAI targeting busy professionals

```

→ User types a free-text prompt. No more y/n questions — the agent decides everything.

```
[1/3] Agent is researching and writing...  
[Memory] Similar approved campaigns found – using as style examples
```

→ Campaign memory searched. Found similar approved posts. Injected as few-shot examples.

```
[Tools] 5/6 · saved 42 tokens  
✅ using : brand_context_tool, news_tool, news_sources_tool,  
generate_content_tool, generate_hashtags_tool  
⏮ skipped: trends_tool
```

→ GPT-4o-mini decided news is useful (prompt mentions AI app, recent context helps) but trends not needed. 42 tokens saved on this run.

```
Done. (5 articles fetched)
```

→ AgentExecutor finished all tool calls. 5 NewsAPI articles were fetched and used.

```
[2/3] Verifying content...  
[Memory] Verifier calibrated with: approved references + denied patterns  
✅ APPROVED – Content is clean and on-brand.
```

→ LangGraph ran verify\_node with memory context. Got verdict JSON. Parsed and printed.

```
[3/3] Review your draft:
```

```
_____  
[full post appears here]  
_____
```

```
Approve and post to Discord? (y/n):
```

→ Human approval gate. The agent has done all its work. Human makes final call.

```
Posting to Discord... ✅  
Saving campaign... ✅  
Updating memory index... ✅  
  
✅ Posted and saved → campaigns/posted_20260518_174201.json
```

→ Discord webhook called. JSON saved. New campaign embedded and added to FAISS index.

---

## 7. Next Plans

### Phase 4 — More tools

- Image generation tool (DALL-E 3) that agent can call when the prompt implies a visual
- Post scheduling tool — generate content now, post at optimal time
- Multi-platform posting — LinkedIn, Bluesky, Email newsletter simultaneously

### Phase 5 — Platform evolution

- FastAPI backend to expose the agent as an HTTP API
- React Flow canvas frontend (visual agent builder, like Flowise)
- Users build their own agent pipelines by connecting nodes on a canvas

### Phase 6 — Advanced memory

- ConversationBufferMemory — remember what happened within a session
- Campaign analytics — track engagement data to feed back into quality scoring
- A/B testing — generate multiple copy variants and score against past performance

### New ideas

- Competitor monitoring tool — scan competitor social accounts before writing
  - Content calendar generator — plan a month of posts from one goal
  - Automatic trend alerts — proactively suggest campaign topics when something goes viral
  - Multi-language support — generate copy in multiple languages simultaneously
- 
- 

## Q&A Preparation — Boss Questions

---

### On LangChain and Agents

**Q: Why use LangChain at all? Your early code called OpenAI directly and it worked fine.**

A: The direct OpenAI calls worked for a fixed sequence — fetch news, generate, post. But a fixed sequence means you always fetch news whether it's useful or not, always generate hashtags the same way, and any change requires rewriting the sequence manually. LangChain's agent pattern lets the model decide what to do based on the prompt. A simple "write about our coffee shop" prompt skips news and trends automatically. A "write based on recent AI news" prompt fetches articles. The agent adapts to intent without code changes. The other benefit is that LangChain gives you the AgentExecutor infrastructure — conversation history management, tool call

handling, error recovery — for free. Building all of that from scratch would be hundreds of lines of code for no product benefit.

---

**Q: The agent "decides" which tools to call — but how? Is it deterministic?**

A: No, it is not deterministic. The agent uses GPT-4o with temperature=0, which makes it nearly deterministic but not guaranteed. The model reads the available tool descriptions and the user's prompt, then produces a tool\_call in its response — a structured JSON specifying which function to call and what arguments to pass. This is OpenAI's function calling feature. The model was trained to pattern-match prompts to appropriate tool signatures based on the descriptions. So the decision quality directly depends on how well the tool descriptions are written. That's why we shortened them in #3a — concise, specific descriptions produce more reliable decisions than verbose ones.

---

**Q: What is a token and why does it matter?**

A: A token is roughly 4 characters of English text, or about 0.75 words. "FlowAI is great" is 4 tokens. Every API call to OpenAI is priced per token — input tokens (what you send) and output tokens (what comes back). The GPT-4o input rate is approximately \$5 per million tokens. This matters because a marketing agent that runs 1000 times per day, with 500 tokens wasted on unnecessary tool descriptions per call, wastes 500,000 tokens per day — around \$2.50 daily or \$900 per year just in unnecessary description overhead. At scale this becomes significant. More importantly, tokens also affect latency — a 200-token prompt responds faster than a 500-token prompt.

---

## On LangGraph

**Q: Why use LangGraph for the verify loop? A while loop would be three lines of code.**

A: A while loop works and was considered. LangGraph was chosen for three reasons. First, state management — in LangGraph, every node receives the full state dict and returns only what changed. There's no risk of accidentally mutating shared variables across iterations, which is a real bug source in complex while loops. Second, traceability — LangSmith traces each graph node as a separate named span, so you can see exactly which revision iteration produced which output. A while loop looks like one monolithic call in the trace. Third, extensibility — when we add a human-in-the-loop approval step inside the graph, or a branch that handles "rejected" differently from "approved", the conditional edge system makes that clean. A while loop with those conditions becomes difficult to read.

---

**Q: What is a StateGraph exactly? What are nodes and edges?**



A: A StateGraph is LangGraph's core abstraction. You define a TypedDict that represents all the information flowing through the pipeline — in our case: content, verdict, issues, summary, revision\_count, max\_revisions, denied\_examples, approved\_examples. Each node is a Python function that receives the full state and returns a dict of fields to update. An edge connects two nodes — it says "after node A runs, go to node B". A conditional edge runs a routing function on the state and returns a string — "approved", "rejected", or "revise" — and each string maps to a different next node. When you call graph.invoke(initial\_state), LangGraph runs the nodes in the order defined by edges until it reaches END.

---

## On Embeddings and RAG

**Q: What is an embedding? Explain it simply, then technically.**

A: Simply: an embedding is a way to convert text into coordinates in a high-dimensional space so that similar meanings end up close together geometrically. "Dog" and "puppy" are close together. "Dog" and "invoice" are far apart. Technically: a neural network encoder (in our case, OpenAI's text-embedding-3-small) processes text tokens and produces a 1536-dimensional floating point vector. The model was trained so that semantically similar texts produce vectors with small cosine distance. FAISS then indexes these vectors using an Inverted File index or Flat L2 structure, and similarity search becomes a nearest-neighbour query in that high-dimensional space — finding the k closest vectors to a query vector.

---

**Q: What's the difference between RAG and fine-tuning? Why did you choose RAG?**

A: Fine-tuning modifies the model's weights by training it on new examples. It's expensive (thousands of dollars), requires hundreds of examples, and produces a new model version that you have to manage and redeploy. If the brand guidelines change, you fine-tune again. RAG leaves the model weights untouched and instead retrieves relevant context at runtime and injects it into the prompt. It's cheap (embedding calls cost fractions of a cent), works with any amount of data, and updates instantly when you change the source document. For brand guidelines that evolve and company data that changes — RAG is the right choice. Fine-tuning would be appropriate if we wanted the model to learn a new writing style at a fundamental level, not just follow provided guidelines.

---

**Q: How does FAISS actually find similar documents? What algorithm does it use?**

A: FAISS (Facebook AI Similarity Search) offers multiple index types. We use the default flat index (IndexFlatL2) for development, which does exact brute-force search — it computes L2 distance between the query vector and every stored vector and returns the k smallest distances. This is 100% accurate but  $O(n)$  per query. For production with millions of vectors, FAISS offers approximate indexes like IVF (Inverted File) which clusters vectors into buckets so search only scans a subset, and HNSW (Hierarchical Navigable Small World) which builds a multi-layer graph for sub-linear search. At our scale (hundreds of campaigns), the flat index is fine.

---

## On Verification and LLM-as-a-Judge

**Q: If the verifier is an LLM, can't it make mistakes? What's your error tolerance?**

A: Yes, absolutely — LLM-as-a-judge has known limitations. It can be inconsistent (same content, different verdict on different runs), biased toward longer content, and fooled by authoritative-sounding but false claims. We handle this in several ways. First, temperature=0 maximises consistency — the model takes the greedy token path every time, so repeated inputs produce near-identical verdicts. Second, we added reference-based calibration in Phase 3 — the verifier doesn't judge in a vacuum, it compares against real approved and denied examples. Third, and most importantly, the human approval gate exists precisely because the automated verifier is not trusted for final decisions. The verifier is a pre-filter that catches obvious problems. The human is the actual quality gate.

---

**Q: Why temperature=0 for the verifier but temperature=0.7 for the content generator?**

A: These are two fundamentally different tasks. Content generation benefits from variability — if you generate the same post every time for the same prompt, the system is useless. Temperature=0.7 introduces controlled randomness so different runs produce different creative outputs. Verification is a classification task — the same content should get the same verdict every time. If the verifier randomly decides "approved" or "needs\_revision" for the same post on different runs, you can't trust it. Temperature=0 makes the model take the highest-probability token at each step, producing near-deterministic output. It's deterministic enough for our use case though not mathematically guaranteed due to floating point non-determinism across hardware.

---

## On Memory

**Q: What's the difference between how memory is used for the copywriter versus the verifier?**

A: They use different search strategies because they have different goals. The copywriter's few-shot examples search by prompt similarity — find campaigns where the user's request sounded similar, regardless of what the final content looked like. The goal is to match creative intent and topic. The verifier's reference examples search by content similarity — find campaigns where the generated text looked similar, regardless of what prompted them. The goal is to compare stylistic and structural quality. A prompt about "AI productivity" might have been written in many different styles. For the copywriter, you want to match the topic. For the verifier, you want to compare the actual text.

---

**Q: What happens to the memory system if someone runs a malicious or low-quality prompt? Does it corrupt the index?**

A: Yes, this is a real concern. If a user posts low-quality content and approves it, that content enters the index as a "positive example" and will be served as a few-shot reference in future runs. This is why the human approval gate is architecturally important — it acts as a quality filter for what enters long-term memory. In production, additional safeguards would include: a quality score threshold before adding to the approved pool, a separate curator role that reviews campaigns before they become training examples, and periodic index audits. This is the same challenge that any RLHF (Reinforcement Learning from Human Feedback) system faces — the quality of the feedback signal determines the quality of the learned behaviour.

---

## On Architecture

**Q: What would break first if this system had to handle 100 users at once?**

A: Three things. First, the FAISS index — we're using a single flat file with no concurrency control. Concurrent writes from multiple users would corrupt it. In production, this would be replaced with a managed vector database like Pinecone, Weaviate, or pgvector. Second, the OpenAI API rate limits — the free tier allows roughly 500 requests per minute. An agent run makes 5-8 LLM calls. 100 concurrent users would generate 500-800 calls per minute, hitting the limit immediately. You'd need to implement a request queue and rate limiter. Third, the `_few_shot_examples` global variable in `ai.py` is not thread-safe. Multiple concurrent users would overwrite each other's few-shot context. This would be fixed by moving to context variables (Python's `contextvars` module) or passing examples as function arguments.

---

**Q: Why GPT-4o for the agent but GPT-4o-mini for content generation and verification?**

A: GPT-4o is used for the agent's reasoning layer — the part that reads tool descriptions, decides what to call, in what order, and how to assemble the final output. This requires strong instruction-following and multi-step reasoning. GPT-4o-mini is 15x cheaper and 3x faster, but slightly less reliable for complex multi-step orchestration. For content generation and verification though, GPT-4o-mini performs comparably — writing a marketing post or evaluating content for safety doesn't require the reasoning depth of orchestrating 6 tool calls. The tool selector also uses GPT-4o-mini because its decision is binary (news yes/no, trends yes/no) and doesn't need heavy reasoning. The cost savings are significant: each run uses maybe 2000 tokens from GPT-4o and 3000 from GPT-4o-mini, roughly 5x cheaper than using GPT-4o for everything.

---

**Q: You mentioned LangSmith. What does it actually show you and how did you use it?**

A: LangSmith automatically captures every LLM call within a LangChain or LangGraph run as a named trace. For one agent run, you'd see a tree: the AgentExecutor at the top, then each tool call as a child node, then the verify StateGraph with `verify_node` and optionally `revise_node` as nested children. For each node you can see: the full input (what was sent to the model), the full output (what came back), token counts, latency in milliseconds, and cost. This is invaluable for

debugging — if the verifier keeps flagging false positives, you can open the trace and see exactly what the model received and why it made that decision. In Phase 3, LangSmith would also let you compare runs before and after adding few-shot examples, to measure whether the quality actually improved.

---

**Q: What is the difference between the brand RAG in `rag.py` and the campaign memory in `campaign_memory.py`? They both use FAISS.**

A: Same technology, different purpose and different data. The brand RAG in `rag.py` embeds `company_data.json` — static brand guidelines, product info, tone rules, approved claims. This never changes unless you update the JSON file. It answers the question: "what does the brand stand for?" The campaign memory in `campaign_memory.py` embeds past campaign runs — user prompts, generated content, hashtags, approval status. This grows after every run. It answers the question: "what have we written before and did it work?" They're also searched with different queries: brand RAG is queried with the user's current prompt, campaign memory is queried differently for the copywriter (by prompt similarity) and the verifier (by content similarity). Keeping them as two separate indexes makes the code cleaner and allows different update strategies.

---

**Q: If you were starting this project over, what would you do differently?**

A: Three things. First, I would structure the services with proper dependency injection from the start rather than using module-level globals for things like `_few_shot_examples`. Globals work for a single-user CLI but become a maintenance problem. Second, I would add proper logging from day one — not just print statements, but structured logging with timestamps and run IDs so you can trace exactly what happened in a specific run days later. Third, I would design the campaign JSON schema more carefully upfront. Early in the project, the schema changed multiple times (adding sources, articles, `verdict_info`), which means old campaign files have missing fields. A versioned schema with defaults would have prevented that inconsistency in the memory index.